

Graduate Algorithms

Problems on Greedy Algorithms

10 September 2026

"I wish it need not have happened in my time," said Frodo. "So do I," said Gandalf, "and so do all who live to see such times. But that is not for them to decide. All we have to decide is what to do with the time that is given us."

— J.R.R. Tolkien

Greedy is a versatile topic, which often gets overlooked due to its supposed simplicity. But as it turns out, a whole lot of problems can be solved by simply being greedy on the right thing (although proving correctness is seldom that straight forward).

I had loads of fun making this and hope you have fun solving it.

§1 Warm Up

☰ Greedy or Not?

Sometimes it can be tricky to tell when a greedy algorithm applies. For each problem, say whether or not the greedy solution would work for the problem. If it wouldn't work, give a counter example.

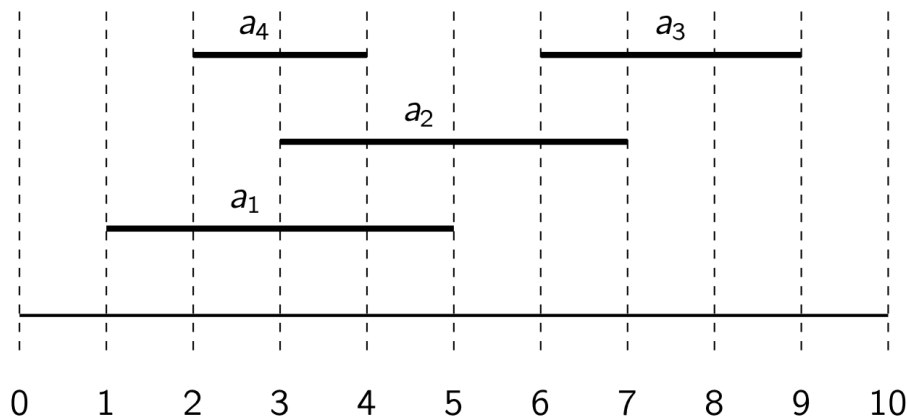
1. You have unlimited objects of different sizes, and you want to completely fill a box with as few objects as possible, or output that it is impossible. (Greedy: Keep putting the largest object possible in for the space you have left)
2. You have unlimited objects of size 3^k for every k , and you want to completely fill a box with as few objects as possible. (Greedy: same approach as the previous problem)
3. You have lines that can fit a fixed number of characters. You want to print out a series of words in a given order while using as few lines as possible. (Greedy: Fit as many words as you can on a given line)
4. There are n hotels in a line, each distance 1 apart and hotel i costing h_i dollars to stay at. You can travel at most distance k every day. Find the minimum total cost of hotels you need to stop at. (Greedy: Go as far as you can before stopping at a hotel)
5. There are r ropes of different lengths, and the cost of connecting two of them is the sum of their lengths. Find a way to connect all ropes into a single rope with the minimum total cost. (Greedy: repeatedly add the smallest two values).
6. Given a weighted graph G and a set of target vertices $T = \{t_1, t_2, \dots, t_m\} \subseteq V$, find the minimum weight path visiting all the vertices of T starting from $s \in V$. (Greedy: Repeatedly go to the closest unvisited target vertex)
7. Given students and projects, each student has projects they are willing to work on. Assign students to projects to maximize the number of students assigned. (Greedy: Assign each student to a currently available project they would like to work on)

§2 Classic Problems

Activity Selection

Given there are n activities $a_1, a_2, \dots, a_n$ with the start and finish time of activity a_i being (s_i, f_i) and doing it means you will be occupied for the whole range $[s_i, f_i]$. Given we start at $t = 0$ and the day ends at $t = T$, give an algorithm that chooses the activities to maximize the number of activities we can do in the day.

For example: $(s_1, f_1) = (1, 5)$, $(s_2, f_2) = (3, 7)$, $(s_3, f_3) = (6, 9)$ and $(s_4, f_4) = (2, 4)$ and $T = 10$. Then, we can do at most two activities: either a_1 and a_3 or a_4 and a_3 .



Hint: Your friend suggests “The best thing to do is to choose the activity (that’s not blocked by previous activity choices) with the smallest finishing time. That way, you get to do an activity and leave as much room as possible for future activities”. Are they correct? Can you prove that?

Job Scheduling

We are given n . Task i takes t_i time and for every hour that passes until task i is done, you pay a penalty cost c_i . Give an algorithm to find the scheduling of tasks that minimizes the total penalty cost.

For example: Consider a city department has the following tasks and say the penalty represents loss in revenue due to the issue.

Name	Time to do	Penalty
Repair a broken bus stop	2 hours	\$ 300/hour
Fill a pothole	1 hour	\$ 200/hour
Clean public park	3 hours	\$ 100/hour

How could they minimize the total penalty? If they fixed in order Bus Stop $\rightarrow$ Pothole $\rightarrow$ Park then the total accumulated penalty would be $2 \times 300 + 3 \times 200 + 6 \times 100 = 1800$ dollars.

However, if we instead do Pothole $\rightarrow$ Bus Stop $\rightarrow$ Park then the accumulated penalty would be $1 \times 200 + 3 \times 300 + 6 \times 100 = 1700$ dollars. We can’t do better than that (you can check).

☰ Scheduling With Lateness

You are given n tasks. The time to do task j (or its length) is l_j and deadline to do it is d_j .

We define the lateness λ_j of a job j in a schedule σ as the difference $C_{j(\sigma)} - d_j$ between the job's completion time and deadline, or as 0 if $C_{j(\sigma)} - d_j$.

Which of the following greedy algorithms produces a schedule that minimizes the maximum lateness? Feel free to assume that there are no ties. In each case, either provide a counterexample to show incorrectness or a brief argument (2-3 sentences) to show correctness.

- (a) Schedule the jobs in increasing order of length l_i .
- (b) Schedule the jobs in increasing order of slack $d_j - l_j$.
- (c) Schedule the jobs in increasing order of deadline d_j .

How will the answer change if, instead of maximum lateness, the goal is to minimize the total lateness $\sum_{j=1}^n \lambda_j$?

☰ Degree Sequence

Given a list of n natural numbers $d_1, d_2, \dots, d_n$, show how to decide in polynomial time whether there exists a simple undirected graph $G = (V, E)$ whose node degrees are precisely the numbers $d_1, d_2, \dots, d_n$. That is, if $V = (v_1, v_2, \dots, v_n)$, then the degree of v_1 should be d_1 , the degree of v_2 should be d_2 , and so on.

☰ Graph Coloring

Given a graph G , color it using $\Delta + 1$ colors where Δ is the maximum degree of a vertex in G , such that if vertex u is connected to vertex v then u and v are colored with different colors.

☰ Prefix-Free Encoding

You are given a set of n symbols, where each symbol i occurs with frequency f_i . We want to encode each symbol as a binary string such that no encoded string is a prefix of another. This allows the encoded message to be uniquely decoded.

The cost of an encoding is the total number of bits required to encode the message. If symbol i is assigned a code of length l_i , then its contribution to the total cost is $f_i l_i$. Thus, the total cost is

$$\sum_{i=1}^n f_i l_i.$$

Give an algorithm that constructs a prefix-free binary encoding with minimum total cost.

§3 Slightly More Fun!

☰ Kyoto Goodwill Event

At the 300th Kyoto Goodwill Event, the students of Tokyo Jitsu High and Kyoto Jitsu High are competing in 1v1 battles.

Each school has N students. The power of the i -th student of Tokyo Jitsu High is $T[i]$ while the power of the i -th student of Kyoto Jitsu High is $K[i]$. In a matchup, the student with higher power wins.

As Tokyo is the organizer, they get to choose the matchups. Give an algorithm that finds a matchup that wins as many rounds as possible.

☰ Assignments and Deadlines

Imagine you have a set of n course assignments given to you today. For each assignment i , you know its deadline d_i and the time l_i it takes to finish it.

With so many assignments, it may not be possible to finish all of them on time. If you finish an assignment after its deadline, you get zero marks. Therefore, you must either complete the assignment by the deadline or not at all.

Give an algorithm to determine the maximum number of assignments you can complete within their deadlines.

☰ Roadtrip

You wish to drive from point A to point B along a highway while minimizing the time you spend stopped for fuel. You are informed beforehand about the capacity C of your fuel tank in liters, your fuel consumption rate F in liters/kilometer, the refueling rate r in liters/minute at which you can fill your tank at a fuel station, and the locations $A = x_1, x_2, \dots, x_{n-1}, x_n = B$ of the fuel stations along the highway.

For example, if you stop to fill your tank from 2 liters to 8 liters, the stop would take $\frac{6}{r}$ minutes.

Consider the following two strategies for refueling:

- (a) Stop at every fuel station, filling the tank with just enough fuel to reach the next fuel station.
- (b) Stop only if you don't have enough fuel to reach the next fuel station; if you do stop, fill the tank to its full capacity.

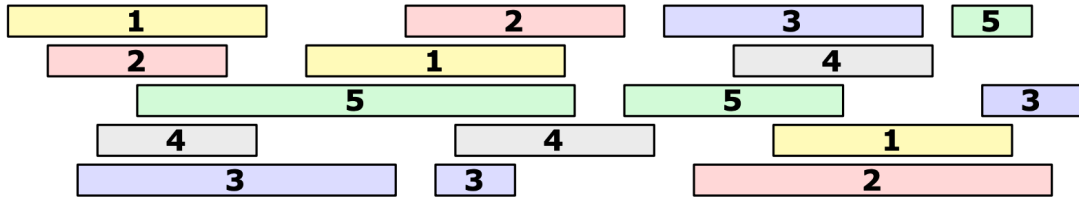
For each strategy, either prove or disprove that it correctly solves the problem.

☰ Coloring Interval Graphs

Let X be a set of n intervals on the real line. A proper coloring of X assigns a color to each interval, so that any two overlapping intervals are assigned different colors.

Describe and analyze an efficient algorithm to compute the minimum number of colors needed to properly color X . Assume that your input consists of two arrays $L[1..n]$ and $R[1..n]$, where $L[i]$ and $R[i]$ are the left and right endpoints of the i -th interval.

For example:



☰ EFX Fair Division

Suppose there are n agents and m items. The value of agent i for item j is given by a nonnegative integer $v_{i,j}$.

An agent's value for a set of items is the sum of its values for individual items in that set.

The goal is to partition the m items among the n agents in a fair manner.

Denote an allocation by $A := (A_1, A_2, \dots, A_n)$, where A_i is the subset of items assigned to agent i .

We require that for any $i \neq k$, $A_i \cap A_k = \emptyset$ (we are partitioning the items) and $\bigcup_i A_i$ is the entire set of items (no item is left unallocated).

An allocation is deemed fair if, for any pair of agents i and k , the value derived by agent i from its bundle A_i is "within an item" of the value it derives from agent k 's bundle A_k ; formally, for every pair of agents i and k and for every item $j \in A_k$, we have that $v_i(A_i) \geq v_i(A_k \setminus \{j\})$, where $v_i(S)$ denotes the value of agent i for a subset S of items.

Design a polynomial-time algorithm for computing a fair allocation when the agents have identical valuations, i.e., item j is valued at $v_j \geq 0$ by every agent (though, for distinct items j and j' , the values v_j and $v_{j'}$ may differ)¹

¹This problem is taken from Prof. Rohit Vaish's tutorial sets (he is a Prof at IIT-D). If you manage to solve this problem for **non-identical valuations**, contact Prof. Rohit.

☰ EFX with Chores

How would your solution to the previous problem change if v_j is allowed to be an arbitrary integer (i.e., negative, zero, or positive)?

Due to the presence of negative values, we need to redefine fairness. An allocation is deemed fair if, for any pair of agents i and k , the value of bundle A_i is “within an item” of the value of bundle A_k .

Formally, for every pair of agents i and k :

- for every item $j \in A_i$ such that $v_j < 0$, we have $v(A_i \setminus \{j\}) \geq v(A_k)$, and
- for every item $j \in A_k$ such that $v_j \geq 0$, we have $v(A_i) \geq v(A_k \setminus \{j\})$.

Your task is to design a polynomial-time algorithm for computing a fair allocation.

§4 Much More Fun!

☰ Glinko's Walk (DS Algo Mock-Midsem 2026)

A heavy rain has swept through the Minaguri Forest, awakening dormant Mushi energy beneath the soil. Ginko has decided to take advantage of the rare rainfall and harvest spirit mushrooms (reishi) that bloom only in such conditions.

Ginko knows a winding trail through the forest containing N glades, each a sacred clearing where spirit mushrooms gather in abundance. Every two consecutive glades are equidistant, and it takes Ginko exactly 1 hour to walk between any pair of adjacent glades.

Upon arriving at a glade, Ginko, very efficient in his craft, collects all the mushrooms there instantly. However, spirit mushrooms, fueled by Mushi energy, regrow in exactly 2 hours after being harvested.

Given the initial number of mushrooms in each glade as an array G and the total duration of Ginko's walk in hours t , given Ginko walk starts at $G[0]$ but can end his walk at any glade along the trail, and may travel in either direction or stop and wait at a glade; write function $\text{max_mushi}(G, t)$ which reports the maximum Mushi Ginko can collect given he takes the optimal path in $O(\min(n, t))$ time.

Input	Output	Explanation
$G = [3, 4, 3, 5, 1]$ $t = 4$	18	$3 \rightarrow 4 \rightarrow 3 \rightarrow 5 \rightarrow 3$
$G = [4, 4, 1, 1, 5, 5]$ $t = 7$	32	$4 \rightarrow 4 \rightarrow \dots \rightarrow 4$
$G = [3]$ $t = 4$	9	We just stand at the Glade.

Hint : Consider the case when $t \gg n$. What do we do then? Can we generalize?

(a) Write pseudocode implementing $\text{max_mushi}(G, t)$ in $O(\min(n, t))$ time.

(b) Argue the correctness of your implementation.

☰ Erina's Ramen (DSCPC E 2026)

Erina has n ramen recipes. The i -th recipe uses broth $A[i]$ as the base. Note: Some ramen recipes may use the same broth.

She is setting up a stall at CWI's annual fest². As the broth takes lot of time to cook and she has only 1 pot, all the ramen recipes on offer at a particular day must use the same broth.

As the kids at CWI are very picky, all the days will have to have pairwise distinct broths.

She may not be able to use all the recipes. It is possible that there are no days for some broths. Finally, the number of recipes on the menu on each day is exactly twice as many as on the previous day and the first day can have arbitrary number of recipes on the menu. This is perhaps as the spoiled CWI kids demand increasing amount of variety.

Your task is to calculate the maximum number of recipes Erina could use throughout the days. Note, that you should not maximize the number of days she can offer.

☰ Faulty Beacons (DSCPC F 2026)

A drone is flying somewhere along an infinite straight runway, but its exact coordinate is unknown.

There are M beacons installed along the runway. Each beacon reports a constraint about the drone's coordinate, but some beacons may be malfunctioning.

Each beacon gives one of the following two types of readings:

- LEFT q : the drone is at a coordinate at most q ,
- RIGHT q : the drone is at a coordinate at least q .

Due to possible failures, not all beacon readings are guaranteed to be correct.

Determine the minimum number of malfunctioning beacons.

☰ Wupendra and Vending Machine (DS Algo Mock-Endsem 2026)

CWI has bought a new vending machine. Unfortunately, as CWI got a 'bargain' on it, the machine is also faulty. It has space for N items (1 indexed). However, when item i is dispensed; item $1, 2, \dots, i - 1$ are dispensed for free. If there are no snacks of some of the types $1, 2, \dots, i - 1$, simply no snack of this type is dispensed. Buying snack of type i is possible only if at least one snack of this type is available.

Prof. Wupendra Wulkarni wants to abuse the machine. The function `optimal_value(C, Q, k)` takes as input an (1-indexed) array C of length N where $C[i]$ is the cost of the i -th item, array Q of length N where $Q[i]$ is the quantity of the i -th item and k , the amount of money Prof. Wulkarni wishes to spend; and returns the maximum worth of items he could obtain.

For example, $k = 8$, $C = [7, 2, 3, 5, 7, 2]$ and $Q = [1, 3, 0, 3, 2, 1]$. We buy a snack of type 6; the vending machine also dispenses one snack of each of the types 1, 2, 4 and 5. We buy a snack of

²Talking about it, we need to rename CMI's annual fest and are still looking for names. If you have suggestions, send it my way!

type 4; in addition to this snack, the vending machine dispenses one snack of type 2. So for a total value of 30.

- Write pseudo-code implementing `optimal_value` in $O(Nk)$
- Argue the correctness of your implementation.

☰ Stationing Sorcerers (DS Algo Mock-Endsem 2026)

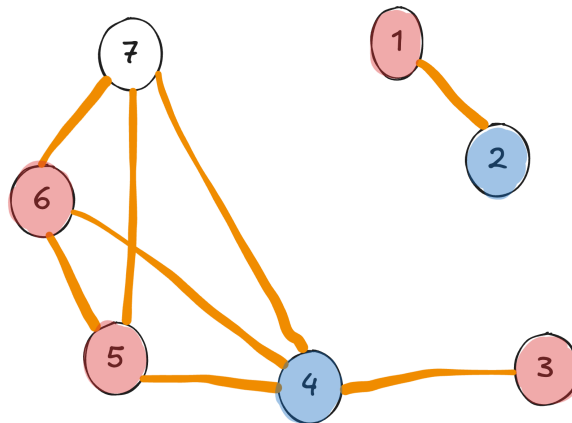
Master Tengen has sensed a massive surge in cursed energy across the n districts of Japan. To maintain the peace, the two rival schools: Tokyo Jujutsu High and Kyoto Jujutsu High, have both requested permission to station sorcerers across the country.

There are n districts in Japan, some of which are connected by high-speed transit routes. To ensure every citizen is safe from Curses, each school (Tokyo and Kyoto) insists that for every single district:

- The district must have a Sorcerer Station from their school, **OR**
- The district must be directly connected to another district that has one.

Master Tengen, however, is wary of the intense rivalry between the two schools. He fears that if a single district hosts stations from both Tokyo and Kyoto, the clashing Cursed Energy will resonate, accidentally birthing a Special Grade Disaster Curse.

See an example below. Red is Tokyo and Blue is Kyoto.



- As a practitioner of the Algorithms Jutsu, you are requested to provide a pseudo-code implementing a function `deployment_plan(G)` that takes the graph with vertices representing the cities of Japan and edges representing connection by a high-speed transit; provides a deployment plan (an array D such that $D[i]$ is 0 if no one is deployed there, 1 if Tokyo High is deployed there and 2 if Kyoto high is deployed there) for the sorcerers of the two schools if possible or returns NO if it is not possible.
- Argue that your algorithm runs in $O(|V| + |E|) = O(n + \binom{n}{2}) = O(n^2)$ time.
- Argue the correctness of your algorithm.

☰ Solo Chess With Rooks

Given a $1 \times N$ grid board with rooks on all the squares with different capturing power $d_i \geq 0$. The only moves allowed are captures where a rook captures another and its capturing power reduces by 1. Once a rook reaches 0 capturing power, it can't move anymore. (a rook can capture another rook if there are no rooks between them).

Devise an efficient algorithm to figure out if there exists a set of moves that leaves only one rook at the end and output the set of moves or declare that it is not possible.³

☰ Discounts

You have n products in your wishlist with prices: $a_1, a_2, \dots, a_n$.

You can either:

- Buy a product individually and pay its full price.
- Use a discount voucher to buy it as part of a group.

You have k vouchers with values: $b_1, b_2, \dots, b_k$

A voucher with value x allows you to select exactly x products. For those x products:

- You pay for the $x-1$ most expensive products.
- The cheapest product is FREE.

Note:

- Each product can be part of at most one discount group.
- Each voucher can be used at most once.

Your task is to find the minimum total cost to buy all products.

☰ Budget Constrained Envy Free Rental Harmony

Given n agents and a house with n rooms. For room j , agent i has non-zero valuation $v_{i,j}$. The total rent for the house is R .

For room j , agent i 's budget is $b_{i,j}$.

We want an bijection σ of agents to room along with a vector $r = (r_1, r_2, \dots, r_n)$, $\sum r_i = R$ where agent i is assigned room $\sigma(i)$ and pays rent $r_i \leq b_{i,\sigma(i)}$.

We call this allocation Envy-Free if there exist no agents i, j such that:

$$b_{i,\sigma(j)} \geq r_j \text{ and } v_{i,\sigma(j)} - r_j \geq v_{i,\sigma(i)} - r_i$$

that is there is no agent who can afford a room not assigned to them and likes it better than their own room.

Give an algorithm to find such an allocation or conclude that no such allocation exists.⁴

³This is a lemma in "Chess is Hard even for a Single Player" by Aravind, Misra and Mittal. Their proof is a bit more complicated, as they missed the simple greedy.

⁴This was solved by me, with Prof. Neeldhara Misra. It was inspired by "Fair Rent Division: New Budget and Rent Constraints" by Gangam, Taherijam and Vazirani

§5 Spanning Trees

☰ Caesar's MST

Professor Caesar has designed a new divide-and-conquer algorithm for computing minimum spanning trees.

For a graph $G = (V, E)$ the set of vertices V is divided into two sets V_1 and V_2 such that $||V_1| - |V_2|| \leq 1$.

Let E_1 be the set of edges that are incident only to vertices in V_1 and E_2 the set of edges that are incident only to vertices in V_2 .

The problem is solved recursively on the two graphs $G_1 = (V_1, E_1)$ and $G_2 = (V_2, E_2)$. Afterwards, one selects the edge in E with minimum weight that crosses the cut (V_1, V_2) in order to connect the two minimum spanning trees into a new spanning tree.

Show or refute: The algorithm computes a minimum spanning tree for G

☰ Not Kruskal's Algorithm?

Kruskal had also considered the following algorithm in his search for an MST algorithm. Does this algorithm work? If it doesn't, explicitly give a graph G where it doesn't give the MST. If it does, why might have Kruskal not 'liked' it?

NOT KRUSKAL'S ALGORITHM

```

1  def not( $G(V, E)$ ):
2      sort  $E$  in decreasing order
3       $i := 0$ 
4      while  $i < |E|$ :
5          edge :=  $E[i]$ 
6          delete(edge,  $E$ )
7          if graph is not connected then:
8              add(edge,  $E$ )
9               $i += 1$ 
10     return  $E$ 

```

☰ Borůvka's Algorithm

Consider the following algorithm:

BORŮVKA ALGORITHM

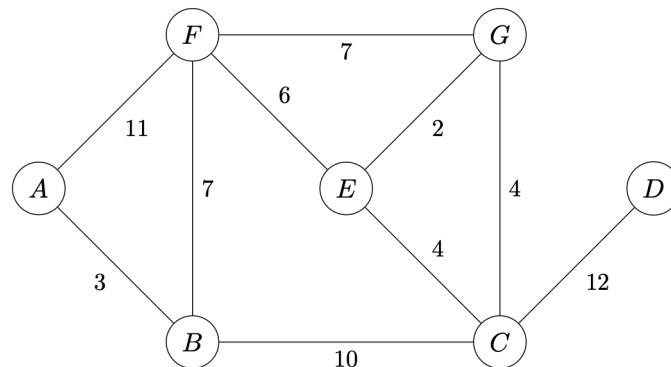
```

1  def bor_al( $G(V, E)$ ):
2      check := [False]  $\times$   $|V|$ 
3      ans := Empty Forest
4      for  $v$  in  $V$ :
5          let  $uv$  be the minimum weight edge incident at  $v$  such that check[ $u$ ] = False
6          then:
7              check[ $u$ ]  $\leftarrow$  True, check[ $v$ ]  $\leftarrow$  True
7              ans = ans  $\cup$   $uv$ 
8       $G' := G$  after contracting all  $e \in$  ans
9      ans' = bor_al(ans)
10     return ans  $\cup$  ans'
```

Does this algorithm terminate? What is the time complexity? Prove or disprove: The algorithm computes a minimum spanning tree for G .

☰ Modifying Kruskal

(a) Compute a minimum spanning tree for the following graph using Kruskal's algorithm. Also provide all other possible minimum spanning trees.



(b) Show that given a graph G and a minimum spanning tree T , the input for Kruskal's algorithm can be adjusted so that Kruskal's algorithm yields T as the result.

☰ Modifying a MST

Suppose we are given the minimum spanning tree T of a given graph $G = (V, E)$ with distinct edge costs, and a new edge $e = uv$ of cost c that we will add to G . Design an $O(|V|)$ algorithm to find the minimum spanning tree of the graph $G + e$.

☰ Minimum Bottleneck Spanning Tree

Consider a connected and undirected graph G with distinct edge costs.

Design a linear-time algorithm to find a minimum bottleneck spanning tree, which is a spanning tree T that minimizes the maximum edge cost $\max_{e \in T} c_e$.

You can assume access to a subroutine FindMedian that, given as input an unsorted list of n numbers, computes the median in $O(n)$ time

§6 Perpetual Scheduling

The problems below are really hard. The former 2 were resolved very recently while the last is still open.

I have included them as I think they represent the state of the art (especially as all of them use greedy).

☰ Bamboo Garden Trimming Problem (Kuszmaul, 2022)

You have a mansion in the country of Pandaland.

There are n bamboos in your garden. The bamboo i grows g_i meters everyday. For sake of convenience, we normalize $\sum_i g_i = 1$.

According to laws of Pandaland, you are only allowed to trim one bamboo everyday (bringing it's length to 0). If the bamboo i is not trimmed for t days, it has height $g_i t$.

You need to declare a height h such that at no point does a bamboo in your garden exceed the height h . All bamboos initially start at 0 length.

Find the minimum height h you can declare as well as provide an algorithm to achieve the same.

Hint: 3 pandas suggested the following algorithms:

1. Chop down the tallest bamboo every day.
2. Choose a threshold x , chop down the bamboo with the fastest growth rate with height above x .
3. Choose a threshold x , chop down the bamboo with height atleast x that will reach h first.

Would any of these work?

☰ Cup Games (Jasińska, Kuszmaul, Lee 2026)

At a frat party, you have n cups in front of you. Every turn, the organizer distributes 1 unit of beer in cups arbitrarily. You then pick a cup and drink atmost 1 unit of beer from it.

Irrespective of the number of turns, find the capacity of cups c you need so that they don't overflow and give an algorithm to choose which cup to drink from.

Hint: The three pandas from above are here (strange) and suggest the following algorithms:

1. Drink from the glass with the most beer.
2. Choose a threshold x , drink from the cup with atleast x units of beer that will reach c first, if the organizer just sticks to the current rate.

3. Choose two thresholds $x < y$, listen to panda 1 beyond x and to panda 2 beyond y .

Would any of these work? What is different in this problem?

☰ Polymerous Scheduling

Given an edge-weighted graph $G = (P, E)$ on a set of people P , where each $e = uv \in E$ represents a relationship with positive growth rate g_e .

Each edge carries a heat score $h_e(t)$ at time t that increases at rate g_e while e is not scheduled and resets to 0 whenever e is scheduled.

A feasible schedule selects, on each day t , a matching $M_t \subseteq E$ (no person can be in two meetings on the same day), and the objective is to minimize the supremum heat over an infinite horizon:

$$\text{heat}(\text{schedule}) = \sup_{\substack{e \in E, \\ t \geq 0}} h_e(t)$$

Using the ideas from above, can you find a schedule that bounds the heat to $4G^*$, where $G^* = \max_{v \in V} \sum_{uv \in E} g_{uv}$ (basically the growth rate of the graph).

Can we do better?⁵

⁵The current lower bound was achieved by me in the process of making this problem set.